

# Chapter 26 - Stage 7 of SKDL -- Executable Knowledge (Part 3): Data Integrity & Comprehensive Architecture

## Completing the Executable Ontology

Chapter 24 built the boundary layer of an executable ontology. Chapter 25 built the value layer. This chapter builds the last layer -- **data integrity** -- and then steps back to view the whole of Stage 7 as a single architecture.

By the end of this chapter, we will have consolidated **five execution primitives** into a single anatomy, examined executability from four complementary perspectives (mathematical, historical, industrial, and EKA), and closed the design-pattern family that began in Chapter 22.

This is the final chapter of Stage 7. It completes the SKDL lifecycle and prepares the synthesis that follows in Chapter 27

- [26.1 Chapter Introduction](#)
- [26.2 Learning Objectives](#)
- [26.3 Exercise 26 -- Cardinality Restrictions and Data Completeness](#)
  - [26.3.1 The Engineering Objectives](#)
  - [26.3.2 Implementation: Creating the Cardinality Restriction](#)
  - [26.3.3 Why the Reasoner Did Not Classify the Three Pizzas](#)
  - [26.3.4 How to Make the Cardinality Fire](#)
  - [26.3.5 Understanding Protégé UI Behavior: Why "SubClass Of" Becomes Empty After Conversion](#)
- [26.4 The Anatomy of an Executable Ontology](#)
- [26.5 Mathematical Perspective](#)
- [26.6 Historical Perspective](#)
- [26.7 Industrial Perspective](#)
- [26.8 EKA Perspective -- Stage 7 Establishes \*Theta\* + \*Phi\*](#)
- [26.9 Engineering Guidelines for Executable Knowledge](#)
- [26.10 Key Concepts](#)
- [26.11 Chapter Summary](#)
- [26.12 Looking Ahead -- Toward SKDL Synthesis](#)

## 26.1 Chapter Introduction

We have now built two layers of an executable ontology.

**The boundary layer** (Chapter 24). Universal restrictions told the reasoner what was *allowed*; closure axioms closed the world around those boundaries. The ontology could classify `MargheritaPizza` and `SohoPizza` as `VegetarianPizza`, deterministically and without ambiguity.

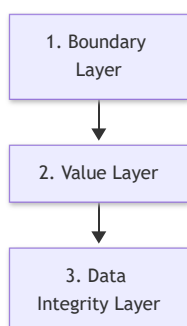
**The value layer** (Chapter 25). Enumerated classes closed the *set of possible values* for a property ( `Spiciness = {Hot, Medium, Mild}` ). `hasValue` restrictions bound a property to a *specific value* from that set ( `JalapenoPepperTopping hasSpiciness value Hot` ). The ontology could fire a trigger on a specific value rather than a category.

But a boundary and a value together are still not enough to execute safely.

Consider what happens when the ontology classifies a pizza as `spicyPizza` because it has at least one `Hot` topping. What if the pizza has *no* topping? What if it has *two* `hasBase` assertions? What if the property that the rule depends on is simply *missing*?

The reasoner under the Open World Assumption will not complain. It will simply *not* fire the rule, and the execution system will silently do nothing. That is the quiet failure mode of an ontology that has not been engineered for **integrity**.

The third layer of executability solves this. It is the **data integrity layer** -- the layer that guarantees the *presence and cardinality* of the data that triggers depend on.



This chapter introduces the last execution primitive -- the **cardinality restriction (Exercise 26)** -- and then consolidates all five primitives into a single architecture.

key Insight:

"A boundary tells the system what is permitted. A value tells the system what to act on. Cardinality tells the system *when it has enough to act*. Only all three together make execution safe."

26.2 Learning Objectives

After completing this chapter, you should be able to achieve the following learning outcomes:

Knowledge:

- Explain why data integrity is the third layer of an executable ontology
- Distinguish the three cardinality restrictions: `min` , `max` , and `exactly`
- Understand how cardinality differs from the closure and value layers
- Explain how the five execution primitives of Stage 7 form a single anatomy
- Trace the two historical lineages of executability: AI reasoning and industrial process
- Apply executable knowledge to industrial scenarios (manufacturing, compliance, process)

Practical Skills:

- Create cardinality restrictions ( `min` , `max` , `exactly` ) in Protégé
- Use `hasTopping min 3 PizzaTopping` and other cardinality patterns
- Convert a class using cardinality to a defined class
- Test cardinality-based classification by synchronizing the reasoner
- Verify that reasoner-inferred classifications match the intended integrity constraints

Mathematical Perspective:

- Understand cardinality as the bounded-extension restriction on a property
- Explain the relationship between cardinality, functional properties, and `hasValue`
- Understand how cardinality restrictions close the "number of values" dimension

Engineering Perspective:

- Understand data integrity as the *last* layer, not the first
- Map the five Stage 7 primitives to the  $\Theta/\Phi$  architecture
- Apply executable ontologies in industrial contexts

26.3 Exercise 26 -- Cardinality Restrictions and Data Completeness

Having established closure in **Exercise 23** and value binding in **Exercise 24 and 25**, we now turn to the final primitive of Stage 7: the **cardinality restriction**..

This exercise follows the tutorial's **Exercise 26**, which introduces `InterestingPizza` -- a pizza with three or more toppings.

26.3.1 The Engineering Objectives

**Exercise 26** introduces the **Cardinality Restriction** -- a restriction on the *number* of values a property may have.

OWL provides three forms:

| Form    | Meaning                  | Syntax                                         |
|---------|--------------------------|------------------------------------------------|
| Minimum | At least <i>n</i> values | <code>hasTopping min 3 PizzaTopping</code>     |
| Maximum | At most <i>n</i> values  | <code>hasTopping max 5 PizzaTopping</code>     |
| Exactly | Exactly <i>n</i> values  | <code>hasTopping exactly 2 PizzaTopping</code> |

A cardinality restriction for an "interesting pizza", for example, would be:

```
Class: InterestingPizza
    EquivalentTo: Pizza and (hasTopping min 3 PizzaTopping)
```

This means: "An `InterestingPizza` is a `Pizza` with at least three toppings."

The engineering objective of **Exercise 26** is to demonstrate how cardinality restrictions **guarantee data completeness** -- the third and final requirement for **execution**.

Why does execution need cardinality?

Consider an execution rule that fires only when a pizza has enough toppings to qualify as "interesting". Without a cardinality restriction, the ontology cannot determine *how many* toppings the pizza has -- it can only say that *some* exist. A rule depends on count cannot fire reliably.

Cardinality closes the *number* dimension. Where the closure axiom closes the *set* of permitted toppings and *hasValue* closed the *specific value*, cardinality closes *how many* values the property may take.

#### Key Insight:

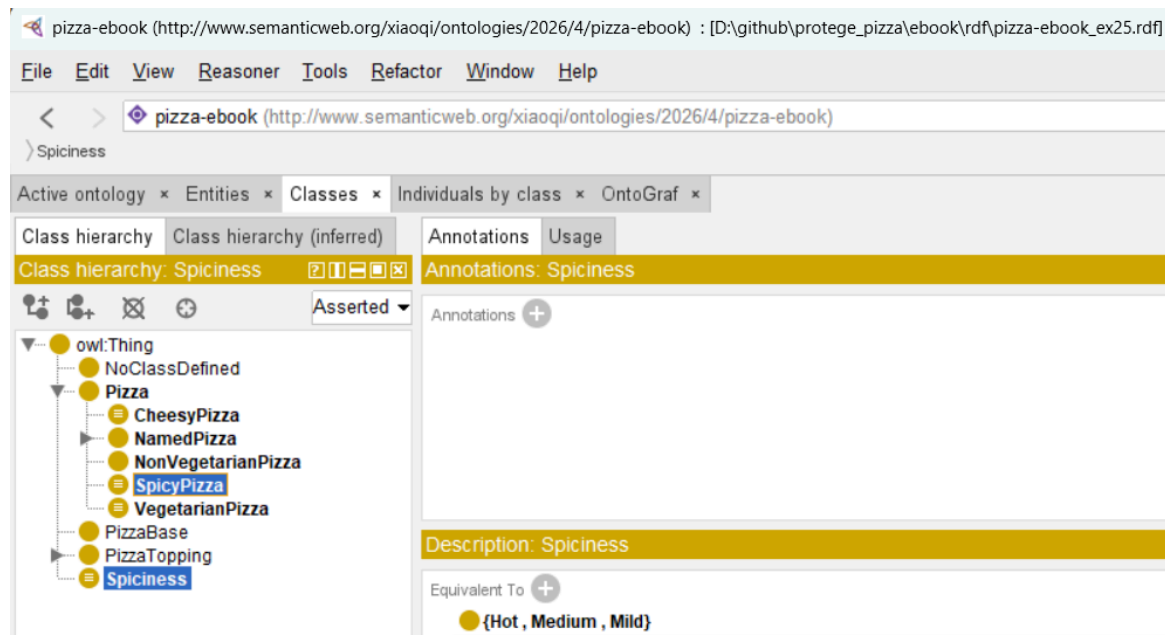
"Cardinality is the last mile of executability. A rule that fires on 'at least three toppings' requires the ontology to know that count is enforced."

## 26.3.2 Implementation: Creating the Cardinality Restriction

Let's walk through the implementation step by step.

### Step 1: Open the `Pizza` ontology

Open `pizza-ebook_ex25.rdf` in Protégé -- the file from the previous exercise, which now contains the enumerated class `Spiciness`, the `hasSpiciness` property, and the defined class `SpicyPizza`.

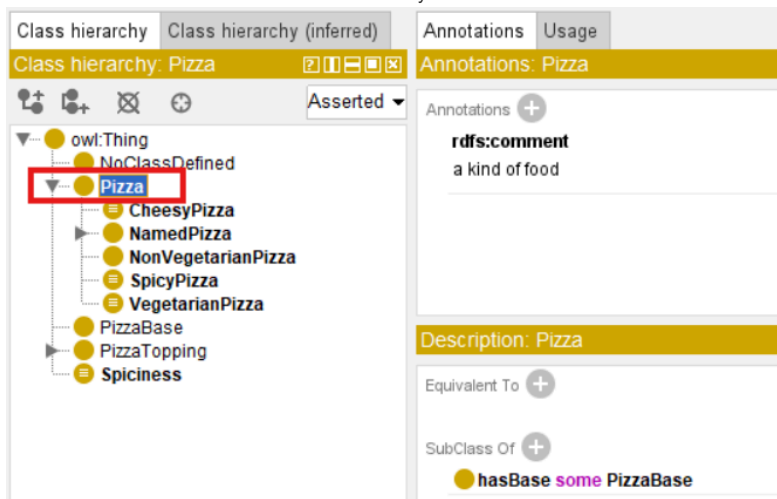


#### Note

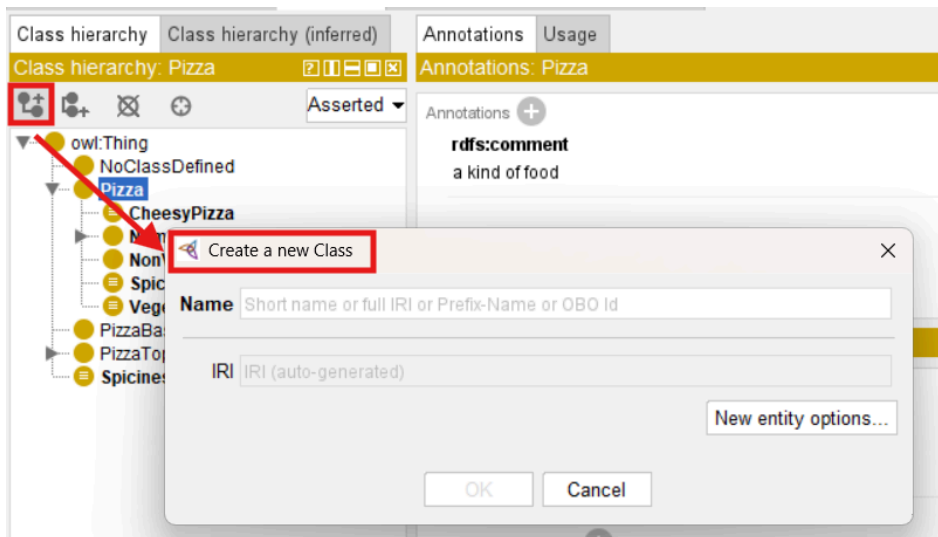
The finished state of **Exercise 26** will be saved as `pizza-ebook_ex26.rdf`.

### Step 2: Create the `InterestingPizza` class

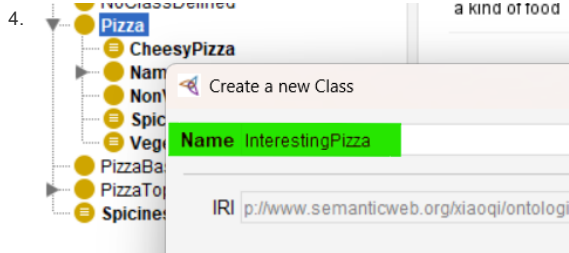
1. Make sure `Pizza` is selected in the class hierarchy



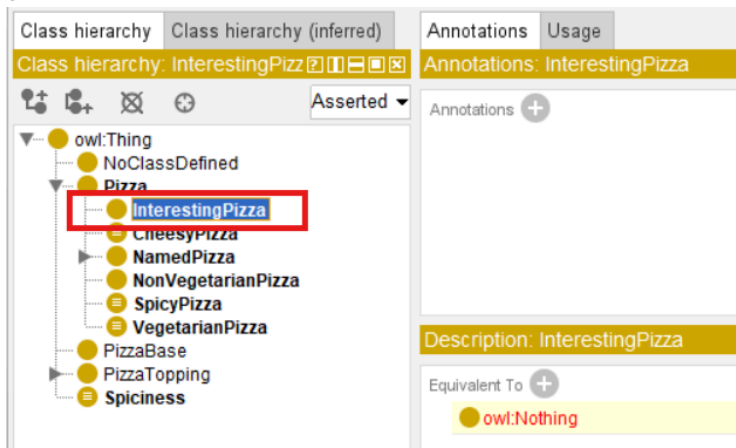
2. Click the **Add subclass** icon



3. Name the new class InterestingPizza

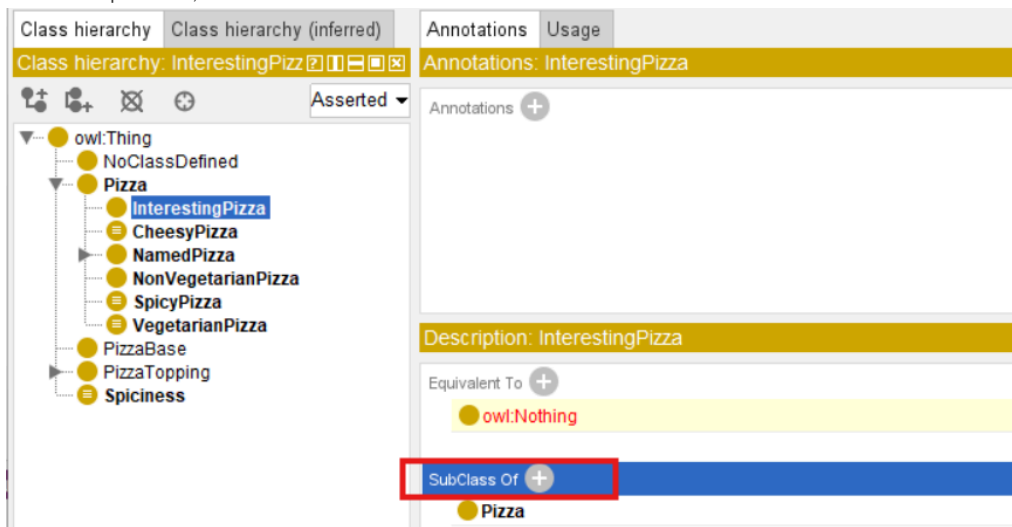


5. Click ok

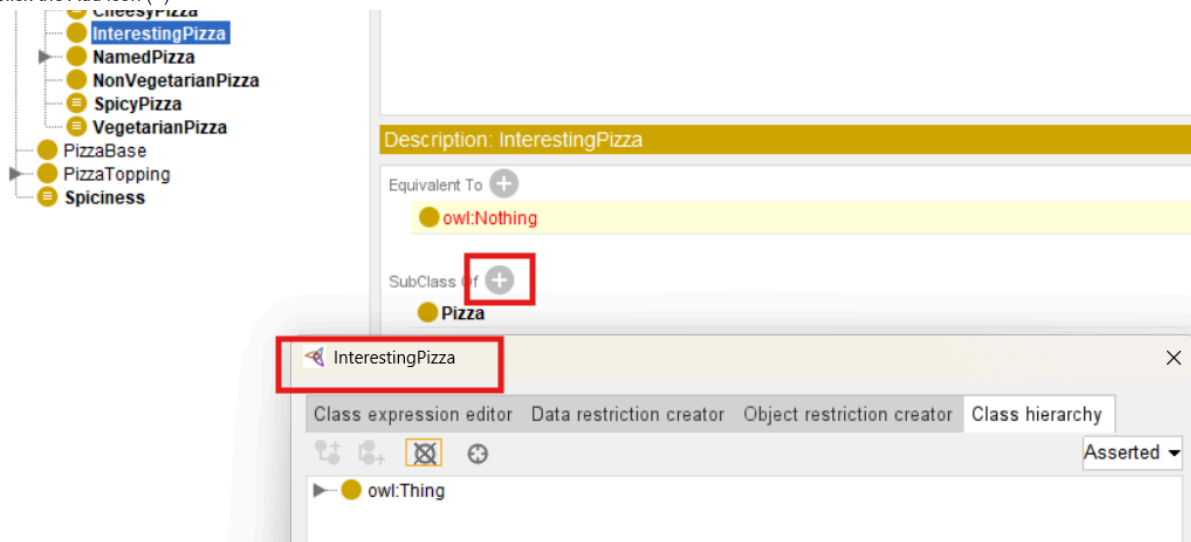


### Step 3: Add the cardinality restriction

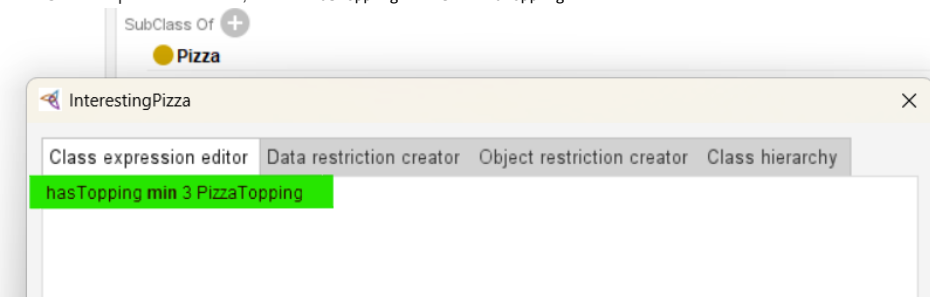
1. Select InterestingPizza (should already in focus after previous step)
2. In the Description view, locate the **SubClass Of** section



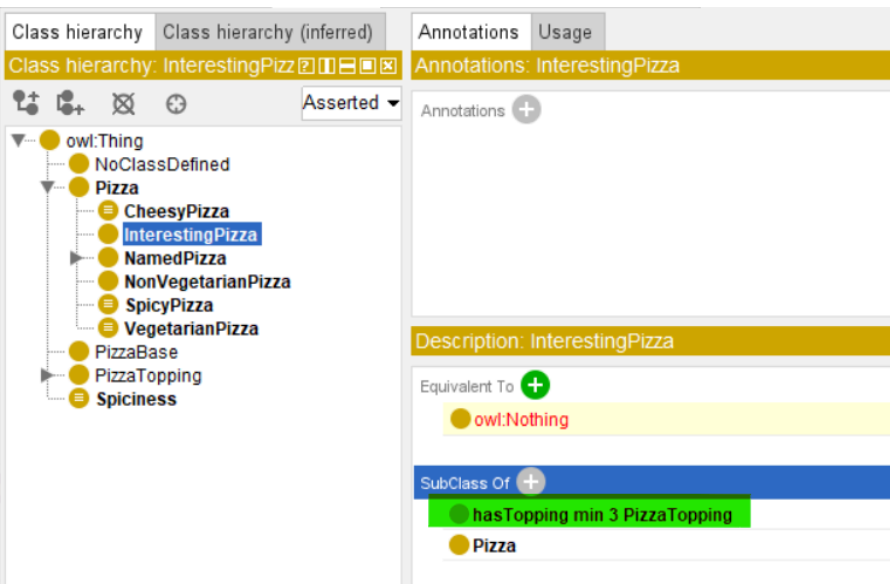
3. Click the Add icon (+)



4. In the Class Expression Editor, enter: hasTopping min 3 PizzaTopping



5. Click ok



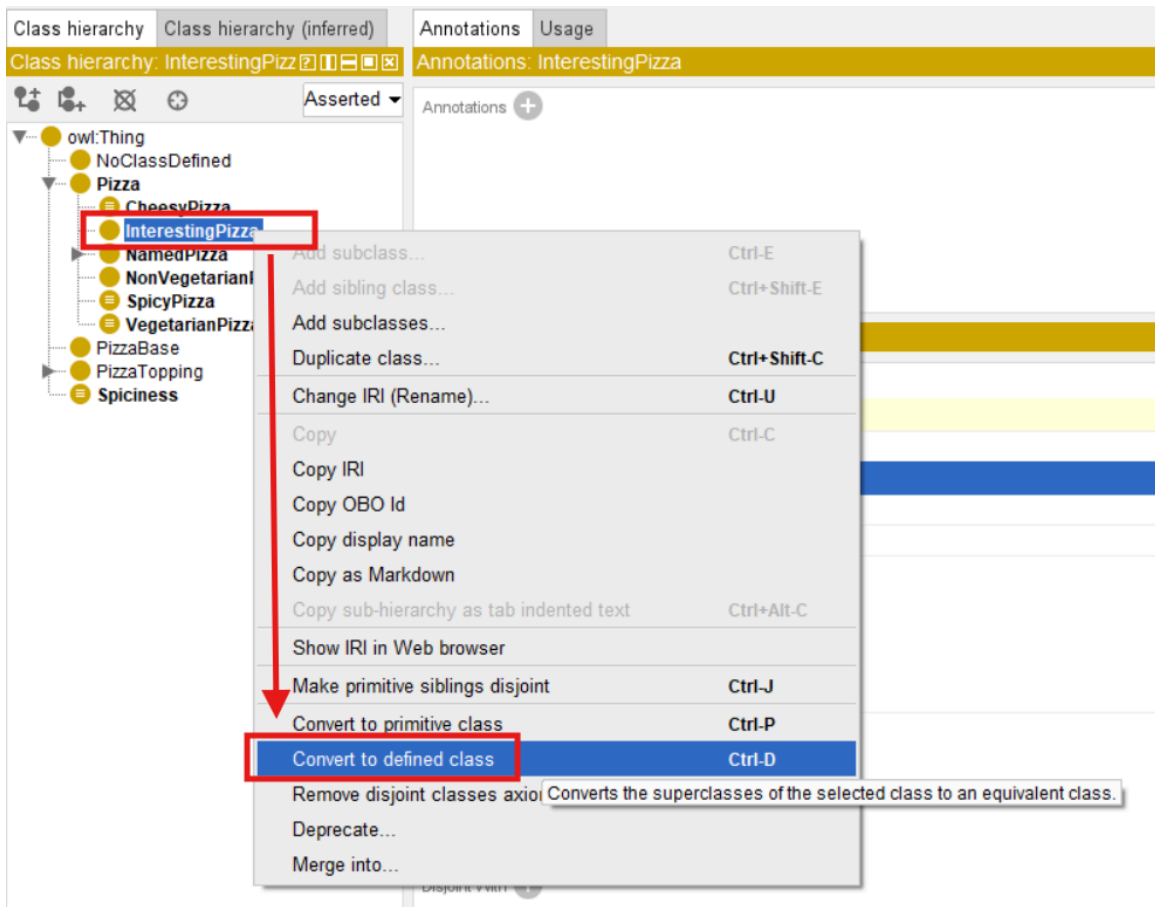
After this step, InterestingPizza has:

```
Class: InterestingPizza
SubClassOf: Pizza
SubClassOf: hasTopping min 3 PizzaTopping
```

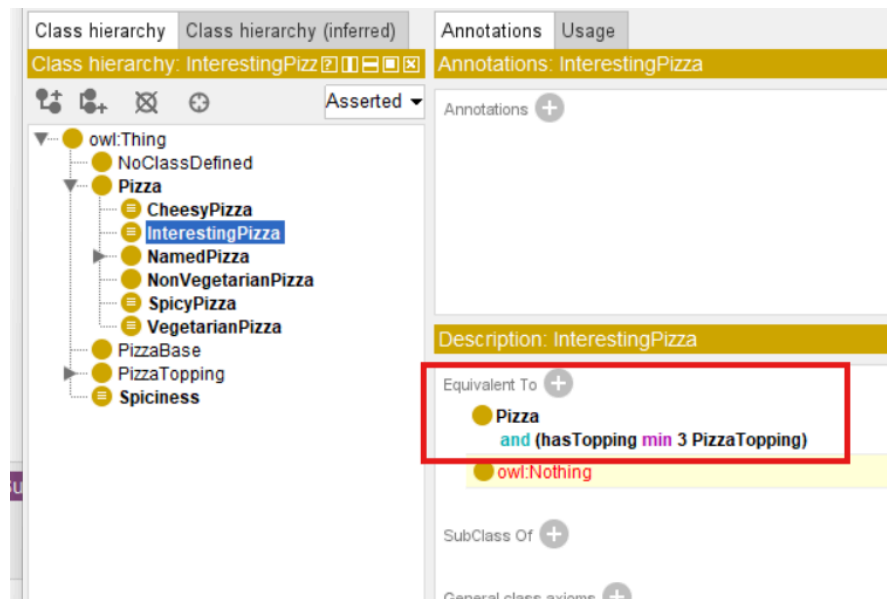
This says: "An InterestingPizza is a pizza and must have at least three toppings."

#### Step 4: Convert InterestingPizza to a defined class

1. Make sure InterestingPizza is still selected
2. Select Edit > Convert to defined class



Observe what happens in the Description view. The `SubClassOf` axiom for the cardinality restriction is moved to `EquivalentTo`:



```
Class: InterestingPizza
  EquivalentTo: Pizza and (hasTopping min 3 PizzaTopping)
  EquivalentTo: owl:Nothing
  SubClassOf: <Empty>
```

#### Note

You may notice the **SubClass Of** section is empty, to learn why, check later §26.3.5 for some clarification.

### Step 5: Synchronize the reasoner

Select Reasoner > Synchronize reasoner (Ctrl+R)

The screenshot shows a software interface with two main panels. The left panel, titled 'Class hierarchy', displays a tree structure starting from 'owl:Thing'. Under 'Pizza', there are several subclasses: 'CheesyPizza', 'InterestingPizza' (highlighted in blue), 'NamedPizza', 'NonVegetarianPizza', 'SpicyPizza', and 'VegetarianPizza'. Below these are 'PizzaBase', 'PizzaTopping', and 'Spiciness'. The right panel, titled 'Annotations', shows the 'InterestingPizza' class. It has a 'Description' section with the text 'InterestingPizza'. Below that, the 'Equivalent To' section shows 'Pizza and (hasTopping min 3 PizzaTopping)'. The 'SubClass Of' section shows 'Pizza'. There are also sections for 'General class axioms' and 'SubClass Of (Anonymous Ancestor)'.

After reasoner synchronized, the **SubClass Of** section contains **Pizza** again.

```
Class: InterestingPizza
  EquivalentTo: Pizza and (hasTopping min 3 PizzaTopping)
  SubClassOf: Pizza
```

#### Step 6: Observe the result

We would assume that the reasoner classifies pizzas with three or more toppings as subclasses of **InterestingPizza**. In the tutorial's data, these include **AmericanaHotPizza**, **AmericanaPizza**, and **SohoPizza**.

However, look at the **Class hierarchy (inferred)** tab:

This screenshot shows the 'Class hierarchy (inferred)' tab. The tree structure is similar to the first screenshot, but now 'InterestingPizza' is highlighted in blue, and it has several subclasses listed below it: 'AmericanaPizza', 'AmericanaHotPizza', 'MargheritaPizza', 'NonVegetarianPizza', 'SpicyPizza', and 'VegetarianPizza'. The 'SubClass Of' section in the right panel still shows 'Pizza'.

You will see something surprising: **Non of AmericanaHotPizza, AmericanaPizza, or SohoPizza are classified as subclasses of InterestingPizza.**

This is not as tutorial mentioned and unexpected. All three pizzas have three or more toppings assertions:

| Pizza             | Toppings                                    |
|-------------------|---------------------------------------------|
| AmericanaPizza    | Mozzarella, Pepperoni, Tomato (3)           |
| AmericanaHotPizza | Mozzarella, Pepperoni, Tomato, Jalapeno (4) |
| SohoPizza         | Mozzarella, Tomato, Olive, Parmesan (4)     |

And yet the reasoner does not classify any of them under `InterestingPizza` .

No worry! This is not a bug in Protégé. It is a subtle consequence of how cardinality restrictions interact with existential restrictions under the Open World Assumption. It deserves its own explanation -- see **§26.3.3**.

### 26.3.3 Why the Reasoner Did Not Classify the Three Pizzas

The behavior in above Step 6 is one of the most instructive moments in this chapter. It looks like a failure, but it is a faithful reflection of OWL's semantics -- and understanding it is essential to writing executable ontologies that work in production.

**The apparent contradiction.** Each of the three pizzas has three or four existential toppings restrictions, e.g. `AmericanaPizza` :

```
Class: AmericanaPizza
  SubClassOf: hasTopping some MozzarellaTopping
  SubClassOf: hasTopping some PepperoniTopping
  SubClassOf: hasTopping some TomatoTopping
```

A human reads this and concludes: "This pizza has three toppings." The `InterestingPizza` definition asks for `hasTopping min 3 PizzaTopping` . The count matches. So why doesn't the reasoner agree?

**The OWL semantics.** Existential restrictions do **not** commit to the number of distinct fillers. `hasTopping some X` means "there exists at least one value of `hasTopping` that is of type `x` ." It does not say "there is exactly one" or "this is the only one of its kind." Under the Open World Assumption, the reasoner is forbidden from assuming the three existentials refer to three distinct individuals unless it can prove it from the axioms.

Even when the filler classes are declared disjoint -- as `MozzarellaTopping` , `PepperoniTopping` , and `TomatoTopping` are -- disjointness establishes that *if* three disjoint individuals exist, they are of different types. It does not establish that three individuals exist. The three existential restrictions could, in the reasoner's view, all be satisfied by the same single individual if that individual happened to be a member of all three classes. (In practice, disjointness rules that out here, but the reasoner cannot bridge from "there exist some X, some Y, some Z" to "there exist at least three individuals" without additional axioms.)

The `min 3 PizzaTopping` restriction, therefore, has nothing to count. The reasoner sees three *existential restrictions*, not three *toppings*, and it cannot satisfy the min-3 cardinality.

**Why this is important?** This is the single most common source of confusion when engineers first write cardinality restriction in OWL. It looks like the ontology should be able to count; semantically, it cannot, without more information. The tutorial's expectation that these pizzas would be inferred as `InterestingPizza` reflects human intuition about the data, not the reasoner's semantics.

This is not a flaw in OWL. It is a *design choice*: OWL refuses to commit to the number of values a property has unless the ontology explicitly closes the world. The consequence for execution is direct: **if a trigger depends on a count, the ontology must be engineered to that the count is provable.**

#### Key Insight:

"The reasoner counts what the ontology proves, **not** what the engineer intends. Existential restrictions prove existence, not count. Only closure or explicit cardinality makes the count provable."

### 26.3.4 How to Make the Cardinality Fire

The fix is not to remove the cardinality restriction. The fix is to give the reasoner enough information to evaluate it.

There are two standard approaches.

#### Approach 1: Add a closure axiom (the pattern from Chapter 24).

Close the world for each pizza class using a universal restriction, so the reasoner knows the pizza's toppings are *exhaustively* the ones listed:

```
Class: AmericanaPizza
  SubClassOf: hasTopping only (MozzarellaTopping or PepperoniTopping or TomatoTopping)
```

The screenshot shows the Protégé interface with the Class hierarchy on the left and the Annotations panel on the right. The Class hierarchy is expanded to show the **AmericanaPizza** class, which is a subclass of **NamedPizza**. The Annotations panel shows the **rdfs:comment** for **AmericanaPizza**: "A pizza that only has Mozzarella, Tomato, and Pepperoni toppings." Below the comment, the **Description: AmericanaPizza** section shows the **SubClass Of** list, which includes **hasTopping some MozzarellaTopping**, **hasTopping some PepperoniTopping**, **hasTopping some TomatoTopping**, **NamedPizza**, and **CheesyPizza**.

A modal window titled **AmericanaPizza** is open, showing the **Data restriction creator** tab. It displays the expression **hasTopping only (MozzarellaTopping or PepperoniTopping or TomatoTopping)** in a green box.

This screenshot shows the same Protégé interface as the previous one, but with a red box highlighting the **SubClass Of** list in the **Description: AmericanaPizza** section. The list now includes **hasTopping only (MozzarellaTopping or PepperoniTopping or TomatoTopping)**, which is the closure of the previous expression. The other subclasses remain the same: **hasTopping some MozzarellaTopping**, **hasTopping some PepperoniTopping**, **hasTopping some TomatoTopping**, **NamedPizza**, and **CheesyPizza**.

With this closure in place, the reasoner knows the pizza has at most the three listed toppings, and (via the existing existential restrictions) at least three. It can now prove the count is exactly 3, and **InterestingPizza**'s **min 3** is satisfied.

The screenshot shows the Protégé interface with the Class hierarchy on the left and the Annotations and Usage panels on the right. The Class hierarchy shows a tree structure starting from `owl:Thing`, with `Pizza` as a subclass. `AmericanaPizza` is a subclass of `NamedPizza`, which is a subclass of `Pizza`. The Annotations panel shows the `rdfs:comment` for `AmericanaPizza` as "A pizza that only has Mozzarella, Tomato, and Pepperoni toppings." The Description panel shows the `Equivalent To` and `SubClass Of` axioms for `AmericanaPizza`.

**Class hierarchy:** AmericanaPizza

**Annotations:** AmericanaPizza

**Annotations:** `rdfs:comment`  
A pizza that only has Mozzarella, Tomato, and Pepperoni toppings.

**Description: AmericanaPizza**

**Equivalent To** +

**SubClass Of** +

- `hasTopping only (MozzarellaTopping or PepperoniTopping or TomatoTopping)`
- `hasTopping some MozzarellaTopping`
- `hasTopping some PepperoniTopping`
- `hasTopping some TomatoTopping`
- `NamedPizza`
- `CheesyPizza`
- `VegetarianPizza`

**General class axioms** +

**SubClass Of (Anonymous Ancestor)**

- `hasBase some PizzaBase`
- `Pizza and (hasTopping only (CheeseTopping or VegetableTopping))`
- `Pizza and (hasTopping some CheeseTopping)`

### Approach 2: Assert the cardinality directly.

Use `exactly` or `min` directly on each pizza class:

```
Class: AmericanaPizza
  SubClassOf: hasTopping exactly 3 PizzaTopping
```

This tells the reasoner the count is exactly 3, without needing a closure. It is more direct but less flexible -- if the pizza's topping set changes, the cardinality axiom must be updated manually.

**Which to use.** In production ontologies, Approach 1 (closure) is generally preferred, because it preserves the pizza's *open* relationship to the rest of the domain and makes the count *derive* from the closed set of toppings. Approach 2 is useful when a pizza's topping count is a defining feature and is fixed by design.

**For this exercise.** Apply Approach 1 to `AmericanaPizza`, `AmericanaHotPizza`, and `SohoPizza`, then re-run the reasoner. All three will now be classified as subclasses of `Interesting Pizza`. The two-topping `MargheritaPizza` will not, because it correctly fails the min-3 threshold.

**This is exactly the kind of engineering decision that distinguishes an ontology that *looks correct from on that executes correctly!*** The reasoner is doing its job; the engineer's job is to give it enough information to do that job reliably.

### Key Insight:

"Cardinality without closure is a promise that reasoner cannot keep. Closure turns intention into provable count."

## 26.3.5 Understanding Protégé UI Behavior: Why "SubClass Of" Becomes Empty After Conversion

When you complete Step 4 and select **Edit > Convert to defined class**, a subtle but crucial change occurs in the Protégé interface that often causes confusion:

- The Mechanics of Conversion:** In Protégé, converting a class to a defined class automatically *moves* the restrictions or superclasses from the **SubClass Of** section up into the **Equivalent To** section. Consequently, the **SubClass Of** box for that class will often appear completely empty immediately after conversion.
- Why This Happens Logically:** A *primitive* class only states necessary conditions (i.e., it is a subclass of something). A *defined* class states necessary *and* sufficient conditions ( `EquivalentTo` ). By shifting `hasTopping min 3 PizzaTopping` into `EquivalentTo`, the reasoner now knows it can *both* recognize pizzas that fit this rule automatically and ensure full logical equivalence.
- Verifying via Reasoner Inferences:** If you notice your axioms have "disappeared" from `SubClass Of`, check the **Equivalent To** field to confirm they safely relocated there. Once you start the reasoner, any class instance meeting this criteria will correctly and dynamically inherit its classification under `InterestingPizza`.

## 26.4 The Anatomy of an Executable Ontology

Syntheses of the five primitives + EKA mapping

## 26.5 Mathematical Perspective

Closure as set restriction, enumeration as finite extension, `hasValue1 as singleton binding, cardinality as bounded extension

## 26.6 Historical Perspective

From expert systems and production rules to modern rule engines and executable knowledge

## 26.7 Industrial Perspective

ISA-95 / Rhize and the reasoning-to-execution pipeline in manufacturing

## 26.8 EKA Perspective -- Stage 7 Establishes $\Theta + \Phi$

## 26.9 Engineering Guidelines for Executable Knowledge

## 26.10 Key Concepts

## 26.11 Chapter Summary

## 26.12 Looking Ahead -- Toward SKDL Synthesis

Last updated at 2026-10-05